An FPGA Based Audio Delay Platform

Venkata Sriram Kamarajugadda Aravindh Nanjaiya Latha Navid Shamszadeh
vkamaraj@pdx.edu aravindh@pdx.edu nshamsza@pdx.edu
Department of Electrical and Computer Engineering
Portland State University
Portland, OR

March 19, 2026

Contents

1 Introduction and Project Description **2**
1.1 Project Description . . . 2
1.2 System Block Diagram . . . 3
1.3 Contributions from Each Team Member . . . 3

2 Hardware Design **4**
2.1 I2S Protocol Overview . . . 4
2.2 I2S2 Top Level Hardware Design . . . 5
2.2.1 I2S2 Receive and Transmit Module . . . 5
2.2.2 I2S2 and Delay Line Clock Domain Synchronization . . . 5
2.3 Delay Line Hardware Design . . . 6
2.3.1 Delay Line Top Module . . . 7
2.3.2 Delay Line Circular Buffer . . . 8

3 Software Design **9**
3.1 State Machine Design . . . 9
3.2 Encoder Input Decoding . . . 10
3.3 Parameter Representation and Hardware Writes . . . 10
3.4 LED and Seven-Segment Display Feedback . . . 10

4 Results and Conclusion **10**
4.1 Usage Statistics . . . 11
4.2 Future Work . . . 11

1 Introduction and Project Description

In the context of audio, delay is an effect that is often used to create echoes. These echoes create a sense of space; for example, echoes spaced 1 or more seconds apart may mimic the response of a large cave. Delays can also be used to implement digital filters, reverbs, and other effects.

From a purely signal processing perspective, if $x[n]$ is the input signal (in our case, digital audio samples), and $y[n]$ is the delayed output signal, then $y[n] = x[n - N]$, where N is the delay length. Intuitively, N can be thought of as how far back in time the signal is delayed, i.e. a larger N, the more delayed the signal is.

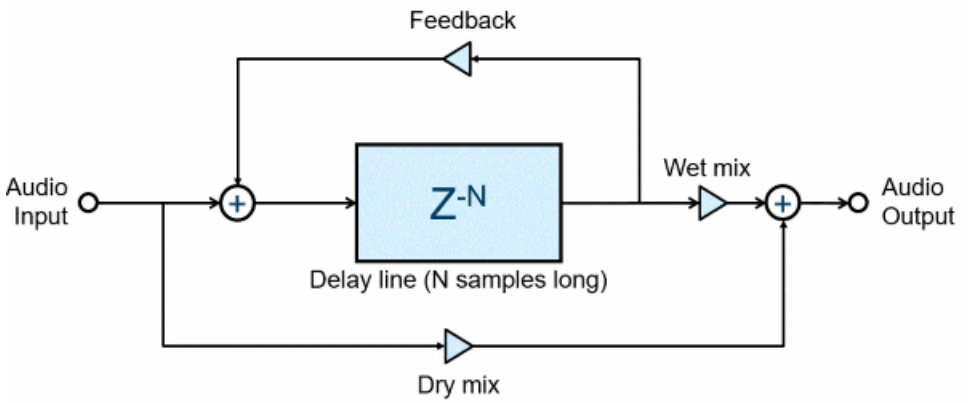


Figure 1: Project Delay Line Diagram

The delay line implemented in this project largely resembles the above diagram. In addition to the actual delay Z^{-N}, there is a scaled feedback loop that sums the delayed signal with the audio input before being fed back into the delay line. There is also a scaled feedforward loop into that mixes the dry (non-delayed) audio signal with the wet (delayed) signal before being sent to the audio output.

1.1 Project Description

This project implements a delay effect that takes an input audio signal, feeds it into a delay line similar to 1, and outputs the original audio signal along mixed with the delayed feedback. Delay length, feedback amount, and dry/wet mix are controlled by the PMOD rotary encoder. Audio IO is done with the PMOD I2S2 peripheral. The project is implemented on the Nexys A7 board loaded with the VeeRwolfX SoC.

Hardware implementations of the I2S Receive and transmit protocols are used to communciate audio data with the PMOD peripheral.

A software state machine controls which parameter the encoder is modifying. The selected delay parameter's value is written to MMIO registers implemented in the delay module.

The delay line itself is implemented in hardware to make use of the FPGA's DSP blocks and low latency. The delay line stores audio in a block-ram circular buffer and outputs a delayed sample to the dry/wet mix and feedback loop. The dry/wet mixer sends the mixed audio to the I2S transmit hardware which routes the audio to the PMOD line out audio jack.

1.2 System Block Diagram

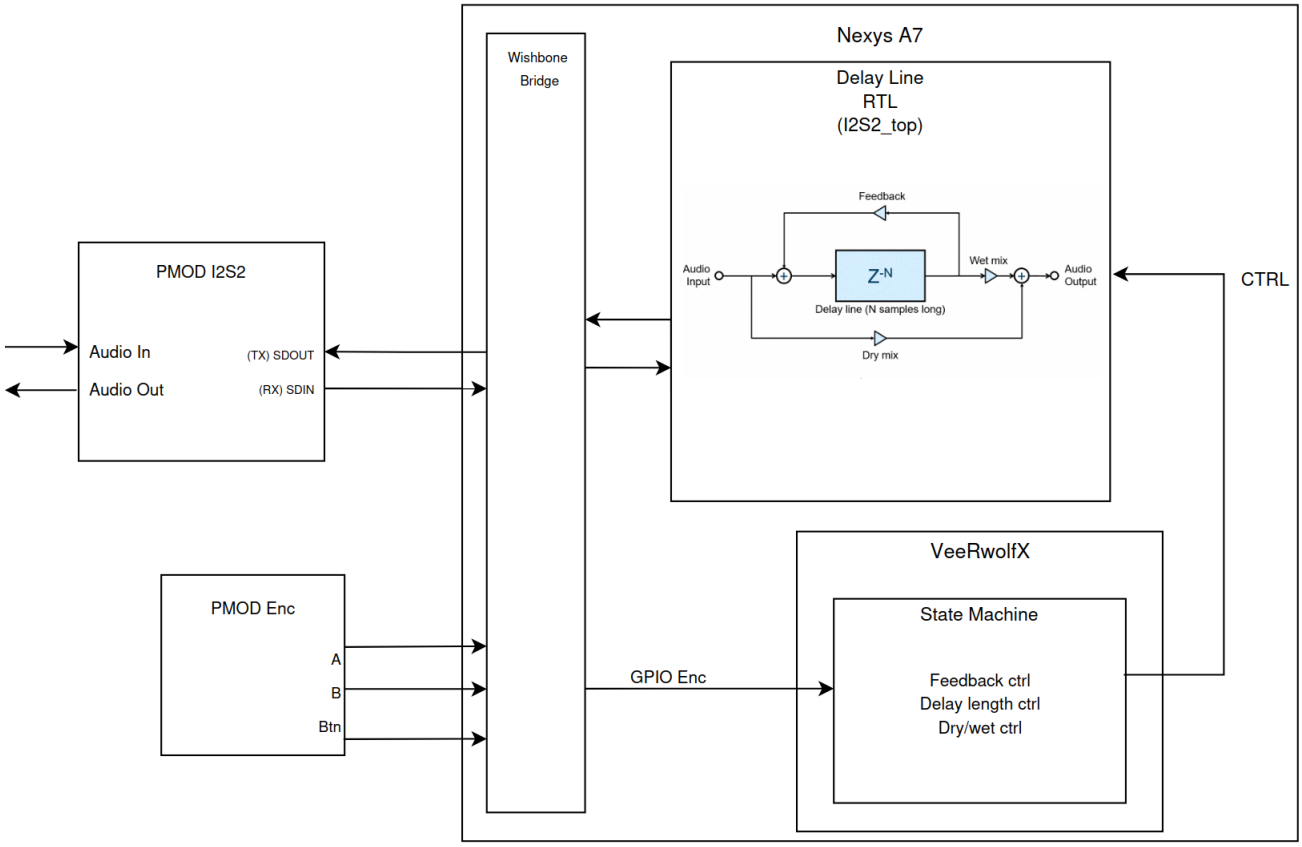


Figure 2: System Block Diagram

The PMOD I2S2 and PMOD Rotary Encoder are both implemented as wishbone compliant peripherals. The Rotary Encoder is a simple GPIO module instantiation. The I2S2 peripheral has its own custom modules defined for audio IO and delay processing. The delay module is in a separate clock domain from the SoC and is interfaced with the SoC via MMIO registers that are synchronized to the I2S clock domain with an enable bit. These MMIO registers are written to by the software running on the RISC-V core and read by the delay module to update the delay parameters. A software state machine running on the RISC-V core controls which parameter the rotary encoder is modifying and updates the corresponding MMIO register along with the enable bit when the encoder is turned. More details on the state machine implementation are given in the software section of the report.

1.3 Contributions from Each Team Member

- Venkata Sriram Kamarajugadda: Delay line RTL design, implementation, and verification.
- Aravindh Nanjaiya Latha: Software state machine implementation, rotary encoder driver implementation.
- Navid Shamszadeh: I2S2 protocol design and implementation, clock generation, and interfacing with the delay line. Software state machine design, hardware delay line design and overall

system integration. Assistance on software implementation and debugging.

2 Hardware Design

The system block diagram in 2 shows the high level hardware design of the system. We present a more detailed description of the hardware design in the following sections. First, the I2S receive and transmit protocol logic is described, then the delay line implementation. Given the audience of this report, we assume the reader is familiar enough with the WISHBONE protocol and the VeeRwolfX SoC design that we do not go into detail about the SoC's interconnect, bus architecture, or how peripherals are integrated into the system. Instead, we focus on the custom hardware modules we designed for this project and how they interface with the SoC and external components.

2.1 I2S Protocol Overview

The Inter-Integrated Circuit Sound (I2S) protocol is a standard for receiving and transmitting digital audio data. It is a synchronous serial communication protocol that uses a controller-worker architecture. The controller generates the clock signals and controls the flow of data, while the worker (in our case the I2S2 peripheral) receives the clock signals from the controller and processes the audio recieve and transmit data accordingly.

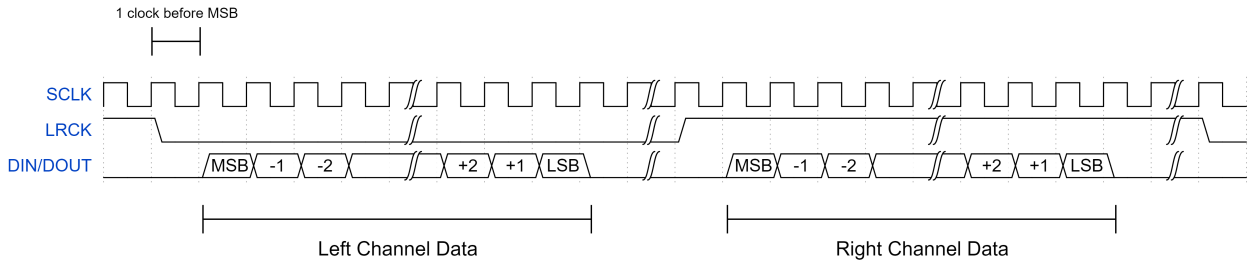


Figure 3: I2S Protocol Diagram

There are technically two protocols that fall under the I2S standard, one for receiving audio data and one for transmitting audio data. The I2S receive protocol defines how the worker receives audio data from an external source, while the I2S transmit protocol defines how the worker sends audio data to an external destination. Both protocols use the same clock signals, but they differ in how the data is processed and transmitted. In our implementation, the I2S2 peripheral is designed to handle both the receive and transmit protocols, allowing it to receive audio data from the PMOD line in jack and transmit processed audio data to the PMOD line out jack.

The clock signals used in the I2S protocol are the bit clock or serial clock (SCLK), the word select or left-right clock (LRCLK), and the master clock (MCLK).

The SCLK is used to synchronize the transfer of individual bits in a given sample, while the LRCLK indicates whether the current audio sample being transferred is for the left or right channel. The MCLK is used to provide a reference clock for the audio processing and is typically a multiple of the SCLK. The MCLK is techinically not a part of the I2S specification; rather it is used to syncrhonize the codec on the PMOD I2S2. In out implementation, the MCLK is also used as the delay line's clock domain.

I2S streams audio data in the following manner: upon an LRCLK transition, the controller waits for a full SCLK period and then starts shifting in (receive protocol) or shifting out (transmit protocol) audio data bits on the falling edge of SCLK. The number of bits in a given sample is

determined by the codec configuration, but in our case it is 24 bits per sample. The controller continues to shift in or out bits until 24 bits have been processed, then the controller waits for the next LRCLK transition, at which point it starts processing the next audio sample. This process continues indefinitely as long as the I2S interface is active. There is technically no handshaking mechanism in the I2S protocol, so the controller and worker must be designed to handle the continuous stream of audio data without any flow control signals.

2.2 I2S2 Top Level Hardware Design

The actual hardware design of the I2S2 peripheral is split into several modules. First, the 22.5 MHz MCLK signal is generated using the same PLLE primitive that generates the SoC clock. The MCLK signal is then assigned to the receive and transmit MCLK pins and also passed into the VeeRwolfX core as one of the input signals to the I2S2 wishbone peripheral.

2.2.1 I2S2 Receive and Transmit Module

The I2S2 peripheral uses the MCLK signal to generate the SCLK and LRCLK signals using a simple counter-based clock divider. The SCLK and LRCLK signals are outputted from the I2S2 top peripheral module to the PMOD I2S2 pins. SCLK is roughly 2.822 MHz (22.5 MHz / 8) and LRCLK (the official sample rate of our audio) is roughly 44.1 kHz (22.5 MHz / 512). The SCLK and LRCLK signals are also fed into the I2S2 receive and transmit logic to synchronize the processing of audio data.

Receive and transmit hardware are implemented in the same module: i2s2.sv. This module is largely based on Digilent's AXI-stream compliant I2S2 design, but modified to interface with the delay line module as opposed to a volume controller. Another key difference is that this I2S2 design is NOT AXI-stream compliant, but rather wishbone compliant with some additional handshaking signals to iterface with the delay line module. Receive and transmit functionality is implemented as separate sections of logic within the same module, but they share the same SCLK and LRCLK signals. Receive logic signals are prefixed with `rx_` and transmit logic signals are prefixed with `tx_`.

The transmit logic handshake signals between the I2S2 module and the delay line module are as follows:

- `tx_valid`: Input port to i2s2 module: indicates that the transmit data is valid and can be processed by the i2s2 transmit logic to stream out to the PMOD line out jack.
- `tx_ready`: Output port from i2s2 module: indicates that the i2s2 transmit logic is ready to accept valid data on the `tx_data` port.
- `tx_last`: Input port from i2s2 module: indicates that the current sample being processed is the last sample in a stereo frame (i.e. the right channel sample).

The receive logic handshake signals are identical but with `rx_` prefixes instead of `tx_` prefixes. The delay line top module interfaces with the I2S2 module using these `valid`, `ready`, and `last` signals to ensure proper timing and synchronization of audio data as it is processed through the delay line and streamed out through the I2S2 transmit logic.

2.2.2 I2S2 and Delay Line Clock Domain Synchronization

The delay line module operates in the same clock domain as the I2S2 module, which is the MCLK domain (22.579 MHz). This allows for simpler synchronization of audio data between the I2S2

module and the delay line module, as they can share the same clock signal. The control parameters for the delay line (delay length, feedback amount, dry/wet mix) are written to MMIO registers by the RISC-V core in the VeeRwolfX SoC, which operates in a different clock domain (12.5 MHz). To synchronize these control parameters to the MCLK domain, we use a simple enable bit that is pulsed by the software after writing new parameter values. The enable bit is synchronized to the MCLK domain using a standard two-flip-flop synchronizer to prevent metastability issues. The delay line module then detects the synchronized enable bit and latches the new parameter values into its own registers on the next MCLK rising edge. This ensures that the delay line parameters are updated synchronously with the audio processing, preventing any timing issues or glitches in the audio output.

2.3 Delay Line Hardware Design

The delay line is the core signal processing module of the FPGA-based audio effects system, designed to implement a real-time audio delay with feedback and dry/wet mixing. The design is implemented in SystemVerilog RTL and operates synchronously with the I2S clock domain.

At the system level, the design is integrated within a Wishbone-based SoC architecture (VeeRwolfX). The control parameters of the delay line—namely delay length, feedback gain, and dry/wet mix—are configured through a Wishbone interface via a state machine. These control values are then provided to the delay line processing modules as synchronized inputs.

Audio data enters and exits the system through the PMOD I2S2 interface. Internally, the delay processing operates on stereo audio streams, which are decomposed into left and right channels and processed in parallel using two identical mono delay line modules.

To optimize FPGA resource utilization, the incoming 24-bit audio samples are reduced to 16-bit values before storage in BRAM. The delay functionality is implemented using a circular buffer, where incoming samples are continuously written into memory and delayed samples are retrieved using pointer offset logic based on a programmable delay length.

A feedback path is implemented to allow recursive echo generation, where a scaled version of the delayed signal is added back into the input. Additionally, a dry/wet mixing stage blends the original signal with the delayed signal to produce the final output.

The design includes pipeline stages to align signal timing with BRAM latency and incorporates a startup mute mechanism to prevent invalid outputs during buffer initialization.

Key Features

- Integration with Wishbone-based SoC (VeeRwolfX)
- Runtime control via Wishbone registers (delay, feedback, mix)
- Stereo audio processing using dual mono delay paths
- Circular buffer implementation using BRAM
- Bit-width optimization (24-bit $\rightarrow$ 16-bit $\rightarrow$ 24-bit)
- Feedback-based recursive echo generation
- Dry/wet mixing using fixed-point arithmetic
- Pipeline alignment for latency matching
- Startup mute using sample counter

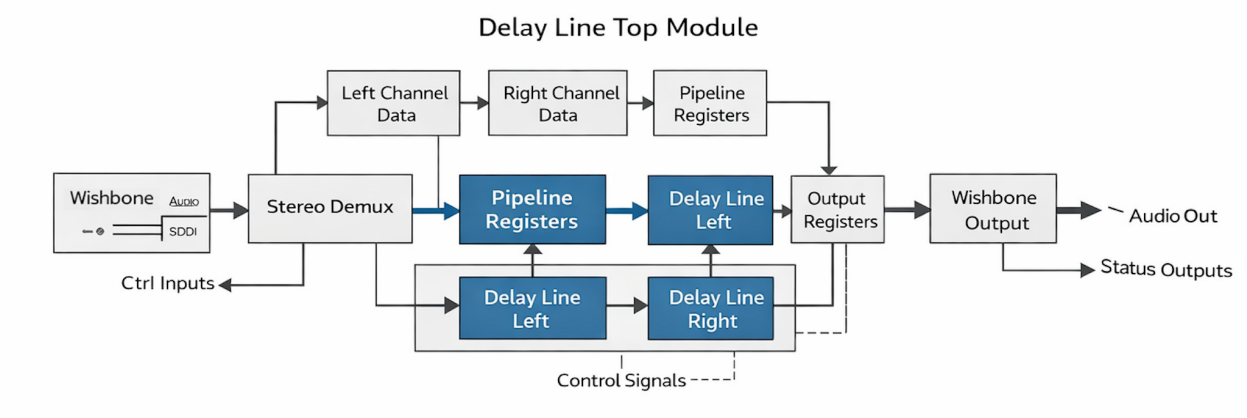


Figure 4: Delay Line Top Module Block Diagram

2.3.1 Delay Line Top Module

The delay line top module acts as the integration block between the audio interface and the delay processing core. It receives stereo audio samples from the I2S interface and processes them through parallel mono delay line instances while being controlled by parameters configured via the Wishbone bus.

The incoming stereo audio stream consists of sequential samples corresponding to the left and right channels. These samples are captured into internal registers (`in_L`, `in_R`) based on channel identification. A complete stereo frame is detected when both channel samples are received.

To ensure proper synchronization, a one-cycle pipeline stage is introduced. The captured samples are registered (`in_L_r`, `in_R_r`) along with a delayed valid signal (`s_new_packet_r`) to guarantee stable inputs for the delay processing modules.

Two instances of the mono delay line (`delay_line_step4`) are instantiated:

- One for the left channel
- One for the right channel

Both instances operate in lockstep, sharing the same clock, the same valid signal, and the same control parameters (`dry_wet`, `feedback`, `delay_len`). This ensures that stereo alignment is preserved without timing skew.

The outputs from both delay modules are captured simultaneously and serialized back into the output stream. The system transmits the left channel first, followed by the right channel, maintaining the original stereo format.

The delay parameters are controlled via the Wishbone interface through a state machine implemented in the VeeRwolfX subsystem. These parameters are continuously fed into the delay modules and take effect dynamically during runtime.

A critical design choice is maintaining continuous input readiness to ensure uninterrupted audio streaming. This avoids deadlock conditions, especially during the startup phase when the delay buffer is still being filled.

Top Module Responsibilities

- Receive stereo audio from I2S interface

- Separate left and right channels
- Pipeline and synchronize input samples
- Instantiate dual mono delay line modules
- Maintain channel lockstep for stereo integrity
- Serialize processed outputs back to stereo stream
- Interface with Wishbone-controlled parameters
- Ensure continuous real-time audio processing

2.3.2 Delay Line Circular Buffer

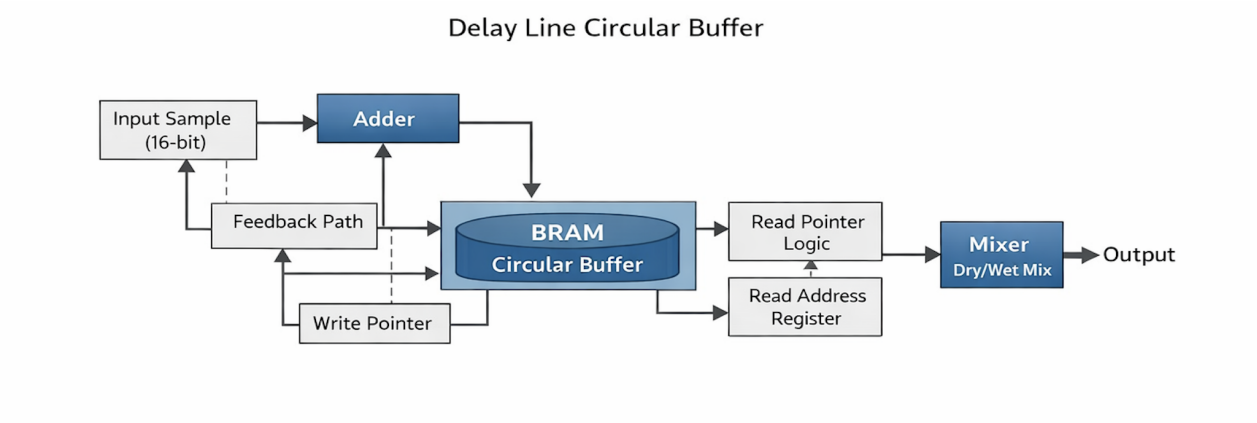


Figure 5: Delay Line Circular Buffer Block Diagram

The circular buffer is implemented within each `delay_line_step4` module and serves as the core memory structure for generating the delay effect. It is realized using FPGA Block RAM (BRAM) for efficient storage and high-speed access.

The buffer operates as a ring memory, where incoming samples are written sequentially using a write pointer (`write_ptr`). Once the pointer reaches the buffer boundary, it wraps around automatically due to its fixed bit width.

The delayed sample is obtained using a read pointer computed as:

$$\texttt{read\_ptr} = \texttt{write\_ptr} - \texttt{delay\_clamped}$$

The delay length is dynamically controlled through a Wishbone-configured register and is clamped to ensure it does not exceed the buffer capacity.

To meet BRAM timing constraints, the read address is registered (`read_addr_reg`) before memory access. This introduces a one-cycle latency, which is compensated by delaying the dry signal path (`dry_pipe1`) to maintain proper alignment.

Each stored sample is represented as a 16-bit signed value, reducing memory requirements while preserving sufficient audio quality.

The buffer supports simultaneous read and write operations, enabling continuous streaming without stalls. A feedback path is implemented by scaling the delayed sample and adding it to the current input before storage, allowing recursive echo generation.

To ensure correct initialization, a counter (`samples_written`) tracks the number of stored samples. Until the buffer is sufficiently filled (i.e., reaches the programmed delay length), the output is suppressed to avoid invalid data.

Circular Buffer Responsibilities

- Store audio samples in BRAM using circular addressing
- Generate delayed samples using pointer offset
- Support simultaneous read/write operations
- Handle pointer wrap-around efficiently
- Clamp delay length to valid range
- Align read latency with pipeline stages
- Enable feedback-based recursive delay
- Prevent invalid outputs during startup

3 Software Design

The firmware is written in C and runs on the RISC-V core of the VeeRwolfX SoC. It implements a polling loop that continuously reads the rotary encoder peripheral and updates the delay line parameters via MMIO writes. The software is organized around three main responsibilities: state machine management, encoder input decoding, and hardware register interaction. Peripheral access is performed using `READ_REG` and `WRITE_REG` macros, which dereference volatile pointers to ensure correct hardware interaction.

3.1 State Machine Design

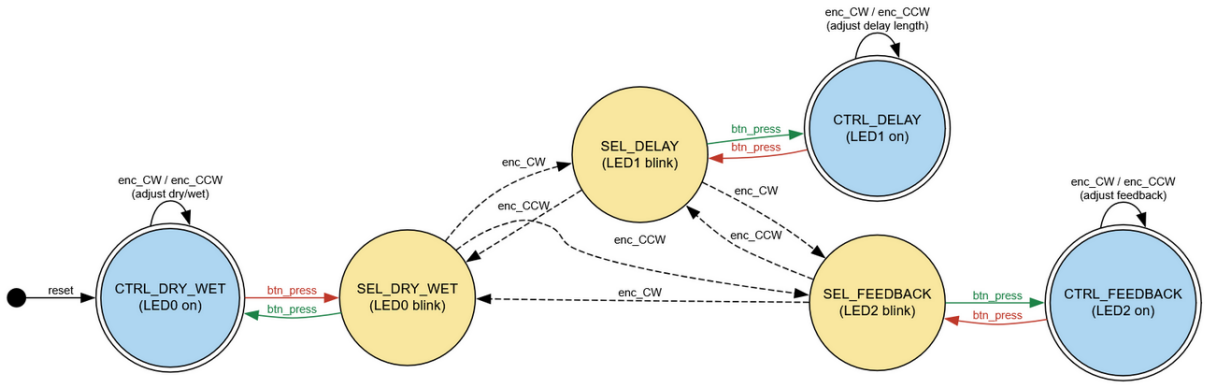


Figure 6: Software State Machine Diagram

The system is modeled as a six-state FSM with two states per parameter—a *control* (CTRL) state and a *select* (SEL) state:

```
typedef enum {
    CTRL_DRY_WET  = 0,  SEL_DRY_WET    = 1,
    SEL_DELAY     = 2,  CTRL_DELAY     = 3,
    SEL_FEEDBACK  = 4,  CTRL_FEEDBACK  = 5
} state_t;
```

In a CTRL state, rotating the encoder increments or decrements the corresponding parameter value and immediately writes the updated value to the delay core's MMIO register. In a SEL state, rotation navigates cyclically between parameters (DRY_WET ↔ DELAY ↔ FEEDBACK), while a button press transitions back into the corresponding CTRL state. Button presses in CTRL states transition to the matching SEL state. This gives the single encoder full control over three independent parameters.

3.2 Encoder Input Decoding

Rotation is detected by monitoring the rising edge of Channel A of the quadrature encoder signal. On each rising edge, Channel B is sampled to determine direction: if A and B differ the rotation is counter-clockwise (increasing value or rightward navigation); if they match it is clockwise (decreasing value or leftward navigation). A short NOP-loop delay of approximately 8000 cycles is applied after edge detection to debounce the signal before re-reading the encoder.

3.3 Parameter Representation and Hardware Writes

All three parameters—dry/wet mix, delay length, and feedback amount—are stored internally as display-domain integers in the range $[0, 100]$ in steps of 5, giving 21 discrete positions. At write time, values are scaled to the hardware register domain:

- **Dry/wet and feedback:** $\texttt{hw} = \texttt{disp} \times 255/100$
- **Delay length:** $\texttt{hw} = \texttt{disp} \times 327$ (samples)

After each parameter write, the `DELAY_UPDATE_EN` register is pulsed to latch the new values into the delay core's clock domain.

3.4 LED and Seven-Segment Display Feedback

Three LEDs indicate the active parameter: LED0 for dry/wet, LED1 for delay, and LED2 for feedback. In a CTRL state the corresponding LED is solid on; in a SEL state it blinks at approximately 4 Hz, implemented via a software counter that toggles the LED every `BLINK_HALF_PERIOD` loop iterations. The eight-digit seven-segment display shows the current parameter's display-domain value (0–100) in BCD format, updated on every iteration of the main loop.

4 Results and Conclusion

The FPGA-based audio delay platform successfully implements a real-time audio delay effect with feedback and dry/wet mixing, controlled via a PMOD rotary encoder. All committed milestones and stretch goals (besides the extremely open-ended last stretch goal) were achieved. The system operates reliably with low latency due to the hardware implementation of the delay line and efficient synchronization between the SoC and the audio processing modules.

The usage of AI tools (Claude, Gemini) was especially helpful when writing testbenches for the various hardware modules, as it allowed for rapid generation of test cases and verification code. The I2S protocol implementation was the most challenging part of the design, as it required clock routing and buffering to meet timing constraints.

4.1 Usage Statistics

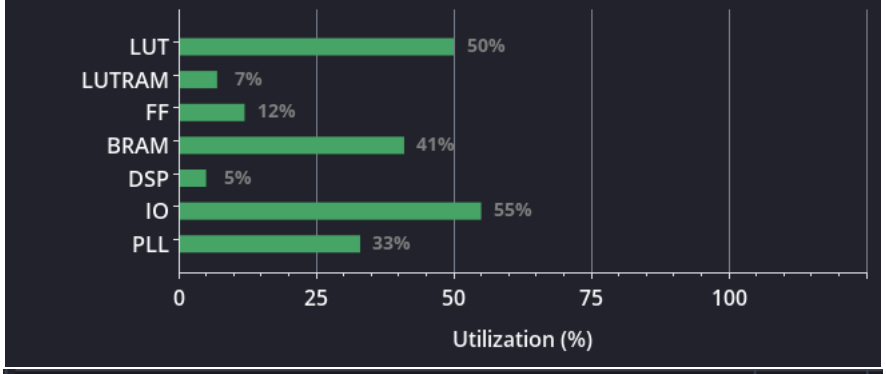
The design ended up utilizing much less FPGA resources than anticipated, though that was after implementing various optimizations such as bit-width and sample rate reduction, which were necessary to fit the design on the available BRAMs. Only 8 DSP slices ended up being used for the feedback gain and dry/wet mix multiplications, which is a very small fraction of the available DSP resources on the FPGA.

4.2 Future Work

The current design is functional and surpasses the original project goals; however, there are several areas which could be massively improved or expanded upon in future iterations:

- **Higher sample rate and bit depth:** The current design operates at a reduced sample rate of 44.1 kHz and a bit depth of 16 bits to fit the delay line within the available FPGA resources. Future work could focus on porting the design to a lighter weight SoC platform, freeing up resources to allow for higher fidelity audio processing.
- **Potentiometers for parameter control:** Instead of using a rotary encoder, potentiometers could be used to provide continuous control over the delay parameters. This would require implementing an ADC interface to read the potentiometer values and map them to the delay parameters. While this is a more complicated design, the user experience would be greatly improved by having more intuitive and responsive controls.
- **Ping pong delay:** The current design processes left and right channels in parallel but does not implement a true ping pong delay effect, where the delayed signal bounces back and forth between the left and right channels. Implementing this would require modifying the delay line to feed the output of one channel's delay into the other channel's input, creating a more immersive stereo effect.
- **Additional effects:** The current design is limited to a simple delay effect, but the architecture could be expanded to include additional audio effects such as reverb, chorus, or flanger. This would involve implementing additional processing modules and integrating them into the audio signal chain, as well as designing a more complex user interface for controlling multiple effects parameters.

Resource	Utilization	Available	Utilization %
LUT	31616	63400	49.87
LUTRAM	1389	19000	7.31
FF	15339	126800	12.10
BRAM	56	135	41.48
DSP	12	240	5.00
IO	115	210	54.76
PLL	2	6	33.33



Name	Used
N rvfpganexys	12
swervolf (veerwolf_core)	12
i2s2_module (i2s2_top)	8
dl_top (delay_line_top)	8
dl_L (delay_line_step4)	4
dl_R (delay_line_step4_1899)	4
rvtop (veer_wrapper_dmi)	4

Figure 7: Top: Overall FPGA resource usage. Bottom: DSP slice usage.